Лабораторна робота №1

Тема: Асинхронна обробка даних у TypeScript

Мета: навчитися налаштувати середовища розробки та набуття практичних навичок асинхронного програмування в TypeScript, зокрема: • встановлення Node.js та налаштування TypeScript-проєкту; • використання Promise.all та Promise.race для керування паралельними операціями; • реалізація механізмів повторних спроб (retry) при помилках; • послідовна обробка даних партіями (batch processing); • коректна обробка помилок через try/catch в асинхронному коді.

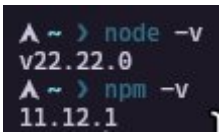
Хід роботи:

3.1. Встановлення Node.js

Встановити Node.js на робочий комп'ютер.

Вимоги:

- Завантажити LTS-версію з https://nodejs.org.
- Переконатися, що команди `node --version` та `npm --version` працюють у терміналі.



3.2. Налаштування проєкту з TypeScript

Створити новий проєкт та налаштувати TypeScript.

Вимоги:

- Ініціалізувати проєкт: `npm init -y`.
- Встановити TypeScript: `npm install -D typescript`.
- Створити `tsconfig.json` через `npx tsc --init`.
- Створити папку `src/` для вихідного коду.
- Додати скрипт `"build": "tsc"` у `package.json`.
- Переконатися, що `npm run build` компілює файли без помилок.

```
term2/nodejs/lab1  main ? > npm init -y
Wrote to /home/lordenko/ztu/course3/term2/nodejs/lab1/package.json:

{
  "name": "lab1",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}


term2/nodejs/lab1  main ? > npm install -D typescript

added 1 package, and audited 2 packages in 1s

found 0 vulnerabilities
term2/nodejs/lab1  main ? > npx tsc --init

Created a new tsconfig.json

You can learn more at https://aka.ms/tsconfig
term2/nodejs/lab1  main ? >
```

```
1 {
2   "name": "lab1",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "build": "tsc",
8     "test": "echo \"Error: no test specified\" && exit 1"
9   },
10  "keywords": [],
11  "author": "",
12  "license": "ISC",
13  "type": "commonjs",
14  "devDependencies": {
15    "typescript": "^6.0.2"
16  }
17 }
```

```
term2/nodejs/lab1  main ● ? ❯ mkdir src
term2/nodejs/lab1  main ● ? ❯ cd src
nodejs/lab1/src  main ● ? ❯ touch index.ts
nodejs/lab1/src  main ● ? ❯ npm run build

> lab1@1.0.0 build
> tsc

nodejs/lab1/src  main ● ? ❯
```

3.3. Функція delay

Сигнатура:

```
delay(ms: number): Promise<void>
```

Створює Promise, що завершується через вказаний час. Використовується як допоміжна функція в інших завданнях.

Вимоги:

- Приймає кількість мілісекунд затримки.
- Повертає `Promise<void>`, що завершується через `ms` мілісекунд.
- Використати конструктор `new Promise` та `setTimeout`.

Встановлення type: “module”, для кращого синтаксису. Також додавання скріпту start

```
{
  "name": "lab1",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "build": "tsc",
    "start": "node ./dist/index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module",
  "devDependencies": {
    "typescript": "^6.0.2"
  }
```

index.ts U

home > lordenko > ztu > course3 > term2 > nodejs > lab1 > src > index.ts > ...

```
interface DelayParams {
    ms: number
}

const delay = async (param: DelayParams): Promise<void> => {
    const { ms } = param

    return new Promise((resolve): void => {
        setTimeout(() => { resolve() }, ms)
    })
}

console.log('Початок');
await delay({ ms: 5000 });
console.log('Готово');
```

```
nodejs/lab1/src  main ? > npm run build

> lab1@1.0.0 build
> tsc

nodejs/lab1/src  main ? > npm run start

> lab1@1.0.0 start
> node ./dist/index.js

Початок
Готово
nodejs/lab1/src  main ? >
```

3.4. Функція `fetchUserProfiles`

Сигнатура:

```
fetchUserProfiles(userIds: string[]): Promise<UserProfile[]>
```

Інтерфейс:

```
interface UserProfile {
  id: string;
  name: string;
  email: string;
}
```

Симулює паралельне завантаження профілів користувачів з сервера.

Вимоги:

- Кожен запит має симулювати затримку 50–150 мс (випадково).
- Використати `Promise.all` для паралельного завантаження.
- Якщо `userIds` порожній — повернути порожній масив.
- Всі змінні та параметри мають бути типізовані.

Приклад використання:

```
const profiles = await fetchUserProfiles(['1', '2', '3']);
// Паралельне завантаження ~100мс, не ~300мс
console.log(profiles);
// [
//   { id: '1', name: 'User 1', email: 'user1@example.com' },
//   { id: '2', name: 'User 2', email: 'user2@example.com' },
//   { id: '3', name: 'User 3', email: 'user3@example.com' }
// ]
```

src > utils > fetchUserProfiles.ts > fetchUserProfiles

```typescript
import { delay } from "./delay.js";

interface UserProfile {
    id: string;
    name: string;
    email: string;
}

interface FetchUserProfilesParams {
    userIds: string[]
}

interface FetchUserProfileParams {
    userId: string
}

const fetchUserProfile = async (params: FetchUserProfileParams): Promise<UserProfile> => {
    const { userId } = params

    const randomMs = Math.floor((Math.random() * 100) + 50);

    await delay({ ms: randomMs })
    return { id: userId, name: `User ${userId}`, email: `user${userId}@example.com` }
}

export const fetchUserProfiles = async (params: FetchUserProfilesParams): Promise<UserProfile[]> => {
    const { userIds } = params

    const promises = userIds.map((userId) => fetchUserProfile({ userId }))
    return await Promise.all(promises)
}
```

```
nodejs/lab1/src  main ? > npm run start

> lab1@1.0.0 start
> node ./dist/index.js

[
  { id: '1', name: 'User 1', email: 'user1@example.com' },
  { id: '2', name: 'User 2', email: 'user2@example.com' },
  { id: '3', name: 'User 3', email: 'user3@example.com' }
]
```

3.5. Функція `retryOperation`

Сигнатура:

```
retryOperation(
  operation: () => Promise<unknown>,
  maxRetries: number
): Promise<unknown>
```

Виконує асинхронну операцію з автоматичними повторними спробами при помилці.

Вимоги:

- При помилці — зачекати 100 мс, спробувати знову.
- Максимум `maxRetries` спроб (за замовчуванням 3).
- Якщо всі спроби провалилися — викинути останню помилку.
- Логувати кожну спробу: `console.log('Спроба ${n}...')`.

Приклад використання:

```
let attempts = 0;
const unreliableOperation = async (): Promise<string> => {
  attempts++;
  if (attempts < 3) {
    throw new Error('Тимчасова помилка');
  }
  return 'Успіх!';
};

const result = await retryOperation(unreliableOperation, 3);
// Лог: Спроба 1... → помилка → чекаємо 100мс
//      Спроба 2... → помилка → чекаємо 100мс
//      Спроба 3... → успіх!
console.log(result); // 'Успіх!'
```

```
import { delay } from "./delay.js";

interface RetryOperationParams {
    operation: () => Promise<unknown>,
    maxRetries?: number
}

let attempts = 0;
export const exampleOperation = async (): Promise<string> => {
    attempts++;
    if (attempts < 3) {
        throw new Error('Тимчасова помилка');
    }
    return 'Успіх!';
};

export const retryOperation = async (params: RetryOperationParams): Promise<unknown> => {
    const { operation, maxRetries = 3 } = params
    let lastError: unknown

    for (let i = 1; maxRetries >= i; i++) {
        console.log(`Спроба #${i}`);

        try {
            return await operation()
        } catch (err) {
            lastError = err
            if (i < maxRetries) {
                await delay({ ms: 100 })
            }
        }
    }

    return lastError
}
```

```
nodejs/lab1/src  main ◉ !? ❯ npm run start

> lab1@1.0.0 start
> tsc && node ./dist/index.js

Спроба #1
Спроба #2
Спроба #3
Успіх!
```

3.6. Функція `processInBatches`

Сигнатура:

```
processInBatches(
  items: number[],
  batchSize: number,
  processor: (batch: number[]) => Promise<number[]>
): Promise<number[]>
```

Обробляє великий масив елементів партіями (batches) послідовно.

Вимоги:

- Розділити `items` на партії розміром `batchSize`.
- Обробити партії **послідовно** (не паралельно!).
- Кожну партію обробити функцією `processor`.
- Повернути масив усіх результатів.
- Логувати прогрес: `console.log('Обробка партії ${n}/${total}...')`.

Приклад використання:

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8];

const processor = async (batch: number[]): Promise<number[]> => {
  await delay(100); // Симуляція обробки
  return batch.map(n => n * 2);
};

const results = await processInBatches(numbers, 3, processor);
// Партія 1: [1,2,3] → [2,4,6] → чекаємо
// Партія 2: [4,5,6] → [8,10,12] → чекаємо
// Партія 3: [7,8] → [14,16] → готово
console.log(results); // [2, 4, 6, 8, 10, 12, 14, 16]
```

src > utils > processInBatches.ts > processInBatches

```typescript
import { delay } from "./delay.js";

interface ProcessInBatchesParams {
    items: number[],
    batchSize: number,
    processor: (params: ProcessorParams) => Promise<number[]>
}

interface ProcessorParams {
    batch: number[]
}

export const processor = async (params: ProcessorParams): Promise<number[]> => {
    const { batch } = params

    await delay({ ms: 500 });
    return batch.map(n => n * 2);
};

export const processInBatches = async (params: ProcessInBatchesParams) => {
    const { batchSize, items, processor } = params

    const totalBatchs: number = Math.ceil(items.length / batchSize)
    const result: number[] = []
    for (let i = 1; i <= totalBatchs; i++) {
        console.log(`Обробка партії ${i}/${totalBatchs}...`)
        const batch = items.slice(i, i + batchSize)
        const resultBatch = await processor({ batch })
        result.push(...resultBatch)
    }

    return result
}
```

```
term2/nodejs/lab1  main ◉ !? > npm run start

> lab1@1.0.0 start
> tsc && node ./dist/index.js

Обробка партії 1/3 ...
Обробка партії 2/3 ...
Обробка партії 3/3 ...
[
   4, 6,  8,  6, 8,
  10, 8, 10, 12
]
```

3.7. Функція raceWithTimeout

Сигнатура:

```
raceWithTimeout(
  promise: Promise<unknown>,
  timeoutMs: number
): Promise<unknown>
```

Виконує Promise з обмеженням часу виконання.

Вимоги:

- Використати `Promise.race`.
- Якщо `promise` не завершився за `timeoutMs` — викинути помилку.
- Текст помилки: `'Operation timed out after ${timeoutMs}ms'`.

Приклад використання:

```
// Швидка операція — успіх
const fast = delay(50).then(() => 'Готово');
const result1 = await raceWithTimeout(fast, 100);
console.log(result1); // 'Готово'

// Повільна операція — timeout
const slow = delay(200).then(() => 'Готово');
try {
  await raceWithTimeout(slow, 100);
} catch (err) {
  if (err instanceof Error) {
    console.error(err.message); // 'Operation timed out after 100ms'
  }
}
```

src > utils > raceWithTimeout.ts > raceWithTimeout

```
import { delay } from "./delay.js";

interface RaceWithTimeoutParams {
    promise: () => Promise<unknown>,
    timeoutMs: number
}

export const fastOperation = async (): Promise<string> => delay({ ms: 50 }).then(() => 'Готово');
export const slowOperation = async (): Promise<string> => delay({ ms: 200 }).then(() => 'Готово');

export const raceWithTimeout = async (params: RaceWithTimeoutParams): Promise<unknown> => {
    const { promise, timeoutMs } = params

    const timeOutPromise = new Promise((resolve, reject) => {
        setTimeout(() => {
            reject(new Error(`Operation timed out after ${timeoutMs}ms`))
        }, timeoutMs)
    })

    return Promise.race([promise(), timeOutPromise])
}
```

```
term2/nodejs/lab1  main ◉ !? ❯ npm run start

> lab1@1.0.0 start
> tsc && node ./dist/index.js

Готово
Operation timed out after 100ms
```

Оформлення index.ts:

```
src > index.ts > ...
import { fetchUserProfiles } from "./utils/fetchUserProfiles.js";
import { processInBatches, processor } from "./utils/processInBatches.js";
import { fastOperation, raceWithTimeout, slowOperation } from "./utils/raceWithTimeout.js";
import { exampleOperation, retryOperation } from "./utils/retryOperation.js";

const main = async (): Promise<void> => {
    console.log('=== Тест 1: Паралельне завантаження ===');
    console.time('fetchUserProfiles');
    const profiles = await fetchUserProfiles({ userIds: ['1', '2', '3', '4', '5'] });
    console.timeEnd('fetchUserProfiles');
    console.log(`Завантажено ${profiles.length} профілів`)

    console.log('\n=== Тест 2: Retry механізм ===');
    const result = await retryOperation({ operation: exampleOperation })
    console.log(result);

    console.log('\n=== Тест 3: Batch обробка ===');
    const items: number[] = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
    const processed = await processInBatches({ batchSize: 3, items, processor })
    console.log('Оброблено:', processed)

    console.log('\n=== Тест 4: Timeout ===');
    const result1 = await raceWithTimeout({ promise: fastOperation, timeoutMs: 100 })
        .catch((err) => err.messaage)
    console.log(result1);
    const result2 = await raceWithTimeout({ promise: slowOperation, timeoutMs: 100 })
        .catch((err) => err.message)
    console.log(result2);
}

await main()
```

Висновок: в ході виконання роботи навчився налаштувувати середовища розробки та набуття практичних навичок асинхронного програмування в TypeScript, зокрема: • встановлення Node.js та налаштування TypeScript-проєкту; • використання Promise.all та Promise.race для керування паралельними операціями; • реалізація механізмів повторних спроб (retry) при помилках; • послідовна обробка даних партіями (batch processing); • коректна обробка помилок через try/catch в асинхронному коді.